Phase 3 Cheat Sheet: Data Collection & DBMS

Sessions 16 – 22 | XML, OLAP, Cassandra, CQL, Data Prep

1. XML & Analytics (Sessions 16 & 17)

OLTP vs. OLAP

Feature	OLTP (Transactional)	OLAP (Analytical)
Focus	Day-to-day ops (CRUD)	Decision making
Speed	Fast, milliseconds	Slow, batch jobs
Data	Current, operational	Historical, consolidated
Example	ATM Withdrawal	Monthly Sales Report

XML Querying & Transformation

Before JSON, XML was the standard. We traverse it using:

- **XPath:** A path language to select nodes.
  - `/store/book` (Selects all books under store)
  - `//author` (Selects authors anywhere in doc)
- **XSLT (Transformation):** Transforms XML into other formats (HTML, Text) for presentation. Essential for ETL.

2. Apache Cassandra (Session 18)

Core Architecture

- **Type:** Wide-Column Store (NoSQL).
- **Topology: Peer-to-Peer Ring** (No Master/Slave).
- **Gossip Protocol:** Nodes exchange state info every second ("I am alive").
- **CAP Theorem:** Partition Tolerance (P) + Availability (A). **Eventual Consistency**.

Cassandra vs. MongoDB

	MongoDB	Cassandra
Model	Document (JSON)	Wide-Column
Master	Primary Node	None (Masterless)
Writes	Moderate	**Extremely Fast**
Lang	MQL (JS)	CQL (SQL-like)

3. Keyspace & Tables (Session 19)

Creating a Keyspace (Database)

Must define replication strategy.

```
CREATE KEYSPACE acts_db
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 1
};
```

Creating a Table

The **PRIMARY KEY** is mandatory.

```
CREATE TABLE users (
    user_id uuid,
    name text,
    city text,
    PRIMARY KEY (user_id)
);
```

Primary Key Anatomy

`PRIMARY KEY ((Partition Key), Clustering Key)`

- **Partition Key:** Determines *which node* stores data.
- **Clustering Key:** Determines *sort order* inside the node.

4. CRUD Operations (Sessions 20-21)

Insert (Upsert Behavior)

Cassandra does **not** check if data exists. It overwrites.

```
-- Generates a new UUID automatically
INSERT INTO users (user_id, name)
VALUES (uuid(), 'Amit');
```

Update & Delete

- **Update:** `UPDATE users SET name='Raj' WHERE id=...` (Needs Primary Key).
- **Delete:** Creates a **Tombstone** marker. Data is removed later during compaction.

Batch Operations

Groups commands atomically.

- **Logged (Default):** Atomic. If node dies, batch replays.
- **Unlogged:** Faster, but not atomic.

```
BEGIN BATCH
   INSERT INTO users ...;
   UPDATE users ...;
APPLY BATCH;
```

5. Data Types (Session 21)

Collections (Complex Types)

Used to store multiple values in one row.

1. **SET:** Unique values, unordered.

```
-- Skills: {'Java', 'Python'} (No duplicates
    )
skills set<text>
```

2. **LIST:** Ordered, allows duplicates.

```
-- History: ['Login', 'Logout', 'Login']
events list<text>
```

3. **MAP:** Key-Value pairs.

```
-- Phones: {'Home': '123', 'Work': '456'}
phones map<text, text>
```

User Defined Types (UDT)

Essential for structured objects (avoiding Joins).

Step 1: Define Type

```
CREATE TYPE address (
    street text,
    zip int
);
```

Step 2: Use in Table (Must be Frozen)

```
CREATE TABLE students (
    id int PRIMARY KEY,
    home_addr frozen<address>
);
```

6. Data Preparation (Session 22)

Data Cleaning

- **Missing Values:** Drop row OR Impute (fill with Mean/Median).
- **Noisy Data:** Remove outliers (e.g., Age = 200).

Feature Encoding

ML models need numbers, not text.

- **Label Encoding:** Low=0, Med=1, High=2 (Ordinal).
- **One-Hot Encoding:** Red=[1,0], Blue=[0,1] (Nominal).